SOFTWARE PROJECT LAB 01 / DEVELOPMENT NOTEBOOK

CloneScope
Phase 4 Full Technical Manual

A source-based explanation of every new module, its functions, algorithmic choices, C++ syntax, integration points, testing evidence, and viva answers

PROJECT	STAGE
CloneScope v0.1 / C++17	Pre-midterm experimental implementation

Prepared for: SPL-01 progress review and board viva
Student: Raian Huq (BSSE1708)
Institute: IIT, University of Dhaka
Supervisor: Dr. Kazi Muheymin-Us-Sakib
Milestone: 7 November 2026

22	66	20
Students	Source files	Body-level candidate pairs

> Central interpretation
> A clone match or a change in coding style is an indicator for human review, never proof of plagiarism. The 22-student fixture is controlled synthetic data, not a real-world accuracy study.

01. What this milestone achieves

CloneScope began with preprocessing and tokenization, moved to Type-1 and restricted Type-2 function-body comparisons, and now incorporates simple, transparent stylometric signals. The code coordinates these features into one final console report for a student cohort.

Delivered in Phase 4	What it means
Original source statistics	Physical lines, blank lines, tokens, recognized function bodies
Identifier style	Counts of unique camelCase, PascalCase, snake_case, lowercase, single-letter, and Other identifiers
Comment style	Comment-containing, comment-only, and inline-comment source lines
Student history	Compare a student's latest numbered assignment with earlier assignments
Detailed clone summary	Each matching pair, both filenames, clone category, and both source-line ranges
Unified final report	Dataset totals, clone counts, style deviations, not-evaluable histories, and review reasons

What remains outside the current scope

- Not yet implemented: Type-3 clone detection, restricted Type-4 screening, CSV/JSON export.
- Not claimed: complete C++ parsing, machine-learning authorship, plagiarism classification, calibrated probabilities, or general real-world precision and recall.
- Next branch: after this documentation and the feature-branch pull request, create a dedicated branch for CSV/JSON export.

02. How the modules connect

The controller is runCloneScope() in src/application.cpp, called by the small src/main.cpp. Every analyzed file has two useful representations: the original text and the comment-cleaned text. Their information must not be mixed up.

Stage	Input → Output	Responsible module
Read & discover	File/folder path → source text and file paths	application.cpp; std::filesystem
Clean	Original source → comment-blanked text	preprocessor.cpp
Tokenize	Cleaned text → typed tokens with line/column	tokenizer.cpp
Recognize	Tokens → supported function-body regions	code_region.cpp, function_region.cpp
Per-file features	Original/cleaned/tokens → source, naming, comment statistics	source_statistics.cpp, identifier_style.cpp, comment_style.cpp

Stage	Input → Output	Responsible module
Student history	Student filenames + statistics → stable history and deviations	assignment_order.cpp, identifier_history.cpp, comment_history.cpp
Match bodies	Two token/region sets → Type-1 and Type-2-only candidates	type1_clone.cpp, type2_clone.cpp
Final evidence	Candidates + aggregates + student findings → report	clone_summary.cpp, analysis_report.cpp

Remember this data fork
Comment analysis uses the original text and the cleaned text. Identifier counting and clone comparison use tokens from the cleaned text. If comments were discarded before measuring comment style, the evidence would be lost.

03. Preprocessing, tokens, and regions

Why comment removal preserves line breaks

The preprocessing scanner replaces recognized comment characters with spaces while retaining newline characters. This allows later tokens to keep meaningful original line numbers. It also handles basic strings and character literals so that markers like // inside ordinary quotations are not mistaken for comments.

```
// original source
int total = a + b; // add two values
return total;
// tokenized after comment removal
int total = a + b;
return total;
```

The token record stores the lexical category, lexeme, line, and column. The region finder identifies matched braces; the function-region module selects supported bodies from those braces. A body's endTokenIndex is exclusive, which helps ensure safe iterator boundaries.

The distinction to explain in a viva

- A physical source line is counted from the original file, regardless of whether it later produces tokens.
- A token represents a lexical item such as an identifier, keyword, operator, or punctuation mark.
- A recognized function body is a token interval selected by a restricted recognizer, not necessarily every possible C++ function.
- A clone candidate is a pair of such bodies that passes a defined comparison rule.

04. Type-1 and restricted Type-2 body matching

Type-1: exact token sequence

After the supported preprocessing, two recognized function bodies qualify as Type-1 when their token counts match and every aligned token has the same type and lexeme. Whitespace and supported comment differences do not affect this comparison.

Type-2-only: bijective identifier renaming

The restricted Type-2 comparator checks the same aligned token structure and all non-identifier lexemes exactly. Identifiers may have different names, but the mapping must remain consistent in both directions; at least one name must differ. Exact Type-1 matches are not counted again as Type-2-only.

Body A	Body B	Current result
`int sum=a+b; return sum;`	`int total=x+y; return total;`	Type-2-only: sum→total, a→x, b→y
`return a+a;`	`return x+y;`	Rejected: a maps to two names
`return a+1;`	`return x+2;`	Rejected: number literal changed

In source files, the matcher evaluates the function body, not the complete function signature. A changed name outside the recognized body alone is not a reason to claim a Type-2 body match. Identifier shadowing, macro expansion and advanced C++ syntax are not fully resolved.

05. Source statistics and naming styles

What one file contributes

Feature	Interpretation
Total lines	Physical lines from original source text
Blank lines	Physically blank source lines
Tokens	Lexical items from supported cleaned source
Function bodies	Recognized supported function-body token regions
Unique identifiers	Distinct spellings, not every usage occurrence
Comment lines	Lines containing supported comment text, measured with original/cleaned source

Identifier naming categories

Style	Example	Historical use
camelCase	calculateTotal	Yes
PascalCase	StudentAverage	Yes
snake_case	total_sum	Yes
lowercase	total	Shown in file totals; no
single letter	x	Shown in file totals; no
Other	_temp, MAX_VALUE	Shown in file totals; no

The convention history purposefully uses only camelCase, PascalCase, and snake_case. It requires enough distinctive names and does not pretend that a student's use of variable x reveals an authorship pattern.

06. Numerical assignment ordering

Directory iterators are not a reliable way to infer chronological assignment order. In lexicographic order, assignment10.cpp can appear before assignment2.cpp. The history module therefore validates each student's filenames, extracts assignment numbers, and numerically sorts them.

Accepted (examples)	Rejected or treated as ambiguous
assignment1.cpp	solution.cpp
assignment_2.cpp	assignment_two.cpp
assignment-10.c	assignment1.txt
ASSIGNMENT3.cpp	Duplicate numbers such as assignment2.cpp and assignment_2.cpp

Once ordered, the highest numbered assignment is the current submission; every lower numbered assignment is historical. Invalid filenames or duplicate numbers cause historical analysis for that student to be skipped, rather than silently selecting the wrong current file. This validation does not prevent the source files themselves from being tokenized and compared for clones.

> A useful board demonstration
> Show assignment1.cpp, assignment2.cpp and assignment10.cpp: despite the console's discovery order, the history output must identify assignment10.cpp as current. Then replace it with solution.cpp to demonstrate the explicit skip warning.

07. Identifier and commenting history rules

Identifier history: consistency before deviation

- Require at least two prior assignments and the current assignment.
- Require at least three convention-bearing unique names in every historical and current sample.
- Each prior sample needs a dominant naming style occupying at least 80% of its convention-bearing names.
- All prior samples must agree on the same dominant naming style.
- Flag a naming deviation only when the current sample has a different clear dominant style.

Comment history: rate and historical stability

For each file, commenting rate = comment-containing lines / nonblank physical source lines. The historical rate is pooled across earlier files, weighted by their nonblank line counts.

- Require at least two earlier samples and 20 nonblank lines in each old/current sample.
- Treat history as stable only when the largest difference between prior per-file rates is at most 10 percentage points.
- Flag current behavior when its rate differs from pooled historical rate by at least 20 percentage points.
- A sudden increase can be flagged just as a sudden decrease can be flagged.

History	Current	Reported outcome
Mostly camelCase across two earlier files	Clearly snake_case	Naming deviation, if evidence thresholds pass
Two old comment rates around 33%	0%	Commenting deviation, if sufficient lines
Earlier styles contradict one another	snake_case	Not evaluable: no stable history
Only one old assignment	Any style	Not evaluable: too little history

> Three states, not merely a binary verdict
> Deviation detected means a threshold rule triggered. Evaluable with no deviation means evidence was sufficient and the rule did not trigger. Not evaluable means insufficient or unstable history; it must never be silently interpreted as normal behavior.

08. The controlled 22-student dataset

The cohort is deliberately synthetic. There are 22 student folders under tests/fixtures/cohort22, each with three standalone C++17 source programs. Every student receives the same assignment brief, but most code is independently varied; selected clone and style cases are injected for deterministic testing.

Submission	Shared assignment brief	Examples of functions
assignment1.cpp	Number-theory exercises	Factorial, GCD, palindrome
assignment2.cpp	Array processing	Selection sort, binary search
assignment3.cpp	Mathematical toolkit	Prime check, Fibonacci, complex addition

Injected clone sets

Injected group	Members	Function	Unordered pairs
Type-1	01, 02, 17, 18, 19	Prime check in assignment3.cpp	10
Type-2-only	05, 11, 12, 13, 14	Fibonacci in assignment3.cpp	10

Each five-person group produces 5 × 4 / 2 = 10 distinct student pairs. These numbers are pairs of matched bodies, not ten independent original programs, not ten guilty students, and not ten plagiarism incidents.

Injected style controls

- Naming changes are controlled for student01, student05, student09.
- Commenting-rate changes are controlled for student05, student09, student20.
- student15 has mixed prior style evidence; its historical assessments are not evaluable.
- Fixture manifests record intended relationships in manifest.csv, expected_clone_pairs.csv, and expected_student_findings.csv.

09. Observed integration output

These numbers were observed in the user's Windows PowerShell integration output after running the combined report against the full cohort22 fixture. They are regression evidence for that controlled dataset, not a general accuracy benchmark.

Measure	Observed
Student folders analyzed / source files processed	22 / 66
Distinct source-file pairs	2,145 = 66 × 65 / 2
Total physical source lines / blank lines	6,973 / 242
Total tokens / recognized function bodies	30,995 / 242
Type-1 / Type-2-only candidates	10 / 10 (20 total)
Naming / commenting deviations	3 / 3
Evaluable naming / commenting histories	21 / 22 each

Measure	Observed
Students with at least one review indicator	12

Location-level evidence: examples from the final report

Type	First student / source lines	Second student / source lines
Type-1	student01 / assignment3.cpp / 4–29	student17 / assignment3.cpp / 4–29
Type-1	student18 / assignment3.cpp / 4–29	student19 / assignment3.cpp / 4–29
Type-2-only	student05 / assignment3.cpp / 34–64	student12 / assignment3.cpp / 32–62
Type-2-only	student12 / assignment3.cpp / 32–62	student14 / assignment3.cpp / 32–62

Line ranges can differ across matched bodies because comments and whitespace affect original source positions. The match operates on the resulting token sequence. The final report provides both line ranges for all candidate pairs, not just an overall candidate count.

10. Understanding the unified analysis report

Section	What the professor sees
1. Dataset overview	Student folders, source files, unordered source-file pairs
2. Source statistics	Combined original lines, blank lines, tokens, function bodies
3. Clone detection	Category totals, affected students, every candidate's two filenames and line ranges
4. Stylometry	Naming/comment deviations, evaluable counts, affected and unevaluable students
5. Human review	Union of students with clone or style signals; reasons listed per student
6. Final report	Evidence locations available and explicit ethical interpretation warning

Why the human-review count is not simply 20

Twenty is the number of matching body pairs, which share students. The final report instead computes a set union of student identifiers across all clone candidates and all qualifying naming/comment deviations. Repeated appearances of one student do not inflate the count.

The twelve students with at least one indicator in the controlled fixture were: 01, 02, 05, 09, 11, 12, 13, 14, 17, 18, 19, 20.

Student	Reported reasons
student01	Type-1 candidate; naming-style change

Student	Reported reasons
student05	Type-2-only candidate; naming-style change; commenting-style change
student09	Naming-style and commenting-style changes; no injected clone
student20	Commenting-style change; no injected clone
Other 8 review students	Clone indicators from their respective injected groups

Classroom ethics
The application must not say that a flagged student plagiarized. A professor should inspect the indicated source lines, assignment constraints, common reference solutions, and independent work history.

11. Build and verification walkthrough

Run these commands from the cloned repository root in Windows PowerShell. Compile all src implementation files; do not compile all 66 fixture programs into the same executable because each has its own main().

Build the full application

```
$sources = Get-ChildItem .\src -Filter *.cpp |
    Select-Object -ExpandProperty FullName
g++ -std=c++17 -Wall -Wextra -pedantic $sources -o clonescope.exe
$LASTEXITCODE   # expect 0
```

Run the cohort and save the large console output

```
@('tests/fixtures/cohort22', 'y') | .\clonescope.exe >
    "$env:TEMP\clonescope_cohort22_output.txt"
Get-Content "$env:TEMP\clonescope_cohort22_output.txt" -Tail 170
```

Dedicated regression programs

Smoke test source	Focus
identifier_style_smoke.cpp	Classification and distinct-name counts
identifier_history_smoke.cpp	Dominant naming history and insufficient evidence
comment_style_smoke.cpp	Line/inline/block comments, literals, empty source
comment_history_smoke.cpp	Rates, thresholds, stability, invalid inputs
assignment_order_smoke.cpp	Numeric ordering, variants, ambiguous/invalid names
clone_summary_smoke.cpp	Pair records, canonical ordering, line boundaries; 7 PASS
analysis_report_smoke.cpp	Combined report structure and indicator sets; 6 PASS

Build each smoke test with its relevant src modules, run the resulting .exe, and verify $LASTEXITCODE equals 0. Rerun earlier preprocessing, tokenizer, region, Type-1 and Type-2 regression tests before merging. The observed run confirms the 22/66 integration totals and the 10+10 clone-body results.

12. Function-call map and code ownership

Question	Correct source-level answer
Where does execution begin?	main.cpp calls runCloneScope() from application.cpp.
Who owns discovery and loops?	application.cpp drives directory traversal, per-file collection, student grouping and pairwise loops.
Who supplies source coordinates?	tokenizer.cpp records token lines/columns; clone_summary.cpp extracts a region's first and last lines.
Where are comparison rules?	type1_clone.cpp and type2_clone.cpp perform restricted body matching.
Where is chronology validated?	assignment_order.cpp validates and sorts numbered assignment filenames.
Where are deviations decided?	identifier_history.cpp and comment_history.cpp apply fixed evidence thresholds.
Who formats final findings?	analysis_report.cpp prints the six-part report, using clone_summary.cpp for candidate locations.
Where do aggregated counts live?	source_statistics.* stores and combines file, token, and candidate counters.

This separation makes the console presentation independent of most detection logic: the report receives already constructed candidate records and student findings. That boundary is the natural future place to add CSV/JSON serialization without rewriting the matchers.

13. Board viva: concise questions and answers

Q01. Why analyze comments before tokenization?

The cleanup deliberately erases comments, so commenting habits must be measured from original source (or original versus cleaned source) first.

Q02. Why are total lines and tokens different?

A line can contain many tokens, only whitespace, or a comment; the measurements represent different units.

Q03. Why unique identifier spellings?

Counting every repeated use would inflate the apparent preference for a repeatedly used variable.

Q04. Why two previous assignments?

A single sample is weak evidence of a stable personal convention; two is a minimum experimental safeguard.

Q05. Why not sort filenames alphabetically?

assignment10 can precede assignment2 alphabetically. Numeric suffix parsing identifies the latest assignment correctly.

Q06. How does Type-2 enforce consistent names?

A forward and reverse identifier map enforce one-to-one correspondence on aligned body tokens.

Q07. Can a new naming style prove copying?

No. Style changes can result from instruction, refactoring, a template, or normal learning.

Q08. Why 2,145 comparisons for 66 files?

The application compares every distinct unordered pair: 66×65/2. It does not compare a file to itself.

Q09. Why 10 matches in a five-student group?

Each pair among five participants counts once: choose(5,2)=10.

Q10. Why only 12 students under review?

A set union removes duplicate student identifiers across the 20 pair records and stylometry flags.

Q11. Why different source lines for a Type-2 pair?

Different formatting and comment placement shift original line numbers, while cleaned body tokens can still match.

Q12. Does zero matches prove unrelated work?

No. It means no recognized body pair passed these restricted Type-1/Type-2 rules.

Q13. What makes the cohort realistic and controlled?

Students share assignment briefs with complete programs, while designated clone/style cases and their expected pairs are documented.

Q14. What is still future work?

CSV/JSON reporting next; Type-3, restricted Type-4 and broader evaluation later.

14. Professor-facing findings and evidence checklist

What has been demonstrated

- The system accepted a 22-student / 66-file recursive folder structure and processed 2,145 file pairs.
- It located the expected 10 Type-1 and 10 Type-2-only body candidate pairs on the controlled assignment3 prime/Fibonacci injections.
- It reported 3 naming and 3 commenting deviations, separately preserving mixed/unevaluable history for student15.
- It combined source statistics and source-line evidence in a final readable report and listed 12 unique students with one or more review indicators.

Demonstration sequence for the review board

Minute	Show	Purpose
0–1	Repository, module overview, clean build	Reproducibility and project scope
1–2	Folder scan and total files/students	Explain input format and grouping
2–4	Type-1 pair and both line ranges	Explain exact token equality
4–6	Type-2 pair with renamed identifiers	Explain mapping and why it is not Type-1
6–8	Naming/commenting deviation and student15	Explain historical baselines and insufficient evidence
8–9	Six-section combined analysis report	Show end-to-end integration and human review
9–10	Limitations and upcoming CSV/JSON branch	Honest scope and roadmap

Before the pull request is merged
Update README.md, add this manual under docs/learning-notes/, rerun relevant tests, review the staged diff, commit and push on feat/statistical-stylometric-analysis, and merge through a reviewed PR. Start CSV/JSON as a separate feature branch from updated main.

15. Glossary and academic caution

Term	Definition
Lexeme	Exact source spelling of a token, for example a variable name or '+' operator
Token	A recognized lexical unit with category, lexeme, line and column
Function-body region	A supported contiguous token interval within a function body
Type-1 body clone	Equal aligned token types and lexemes
Type-2-only body clone	Equal structure and nonidentifier lexemes, with consistent identifier renaming
Stylometry	Description of observable code-writing habits, such as naming and commenting style
Historical baseline	The consistent style/rate of a student's earlier numbered assignments
Not evaluable	The required history, stability or sample size is missing
Synthetic fixture	Artificially constructed test dataset with a known expected outcome
Human-review indicator	A prompt to inspect evidence, not an automatic judgment

Project position: CloneScope is an explainable, rule-based student-work analysis prototype. Its strongest evidence is the traceable connection between a reported candidate, the exact matching rule, two original source locations, and the student's historic style information. Its most important boundary is ethical: none of those indicators, alone or together, constitutes proof of plagiarism.

PART II / ANNOTATED SOURCE WALKTHROUGH

From individual functions to the entire program

This part is meant to be studied with Visual Studio Code open. File names, function names, data fields, thresholds, and code excerpts come from the current feat/statistical-stylometric-analysis branch. The source excerpt numbers indicate real line numbers in the examined branch snapshot; subsequent edits may shift them.

> Reading method
> For each module, follow four questions: (1) What data is received? (2) Which exact conditions and C++ functions process it? (3) What is returned or changed? (4) Where is that result consumed next? This is the foundation for explaining the architecture confidently in the viva.

16. Exact feature-branch change inventory

The GitHub comparison of main with feat/statistical-stylometric-analysis showed 14 commits ahead, zero behind at the time this manual was prepared. The feature does not replace the earlier detector; it extends its pipeline and output.

Change class	Files and purpose
Existing controller modified	src/application.cpp: integrates all new measurements, student chronology, pair evidence and final output
Eight new feature modules	source_statistics, identifier_style, identifier_history, comment_style, comment_history, assignment_order, clone_summary, analysis_report; each has a .hpp interface and .cpp implementation
Seven new smoke tests	identifier_style, identifier_history, comment_style, comment_history, assignment_order, clone_summary and analysis_report
Ordering fixtures	assignment_order/student1 and assignment_order_invalid/student1, including assignment10 and unsupported solution.cpp
Controlled cohort	tests/fixtures/cohort22: 22 student folders, 66 independent C++ files, README and four CSV control files

Why separate .hpp and .cpp files?

A header (.hpp) announces the public interface: structures, enum types, function signatures. A source file (.cpp) defines what those functions actually do. application.cpp includes the headers to call the functions without copying their implementations into one huge file. The linker's job is to combine the separately compiled implementations.

- For example, identifier_style.hpp declares classifyIdentifierStyle(...), while identifier_style.cpp implements the classification checks.
- Header include guards via #pragma once prevent the same header from being processed repeatedly in a translation unit.
- The 66 cohort assignments are test inputs, not library modules to link into clonescope.exe; they each contain a main() function.

17. C++17 language and library syntax you must know

C++ expression	Meaning in this project
std::size_t	Unsigned size/index/count type used for token counts, line counts and array positions
const std::string &s;	Read-only reference to a string, avoiding a full string copy
std::vector<T>	Ordered, resizable collection: tokens, region lists, analyzed files and match entries
std::map<K,V>	Sorted key/value collection: student groups and numeric assignment suffixes
std::unordered_set<T>	Hash-backed uniqueness check: a unique identifier's spelling is counted once
std::set<std::string>	Sorted unique student/reason values: prevents duplicated review-indicator participants
enum class	Type-safe named category, e.g. CamelCase or CloneCandidateKind::Type1
constexpr	Compile-time constant, e.g. dominanceThreshold = 0.80
namespace { ... }	Internal linkage: helpers are private to that .cpp file
std::ostringstream	Build strings or capture report output using stream insertion
std::istringstream	Treat an existing source string as a readable line stream
std::filesystem::path	OS-aware paths, extensions, relative subfolders and filenames
std::tie(a,b)	Compare multiple fields in a deterministic lexicographic order

Syntax examples used by the feature

```
for (const Token &token : tokens) { /* read each token */ }
if (!seenIdentifiers.insert(token.lexeme).second) continue;
const auto &student = *studentFileIndices.begin();
std::lround(rate * 100.0);  // round a fraction for display
```

- In for (const Token &token; : tokens), each element is accessed by a read-only reference. The compiler infers the value type in auto declarations.
- The insert(...).second boolean from a set tells whether the value was new; false means this identifier spelling was already encountered.
- A C++ lambda such as [](unsigned char c) { return !std::isspace(c); } is a small unnamed function supplied to a library algorithm.
- Always cast a possibly signed char to unsigned char before calling cctype classification functions such as isspace, isalpha or isdigit.

18. source_statistics: counting original lines and totals

Public data structures

Record / field	Exact purpose
SourceStatistics.totalLines	Count of physical lines read from original text
SourceStatistics.blankLines	How many of those lines contain only whitespace
SourceStatistics.tokenCount	Token count supplied by the tokenizer
SourceStatistics.recognizedFunctionBodies	Region count supplied by function-region detection
AggregateStatistics.fileCount	Number of successfully analyzed files accumulated
AggregateStatistics totals	Sums of line, blank, token and recognized-body counters
AggregateStatistics.type1CandidateCount / type2OnlyCandidateCount	Counters incremented later by application.cpp after per-file pair matching

calculateSourceStatistics(sourceCode, tokenCount, recognizedFunctionBodies)

This function receives the raw original text and two computed counts. It creates an istringstream, reads lines with getline, increments totalLines, then scans characters on each line. If every character is whitespace, it increments blankLines. The function does not tokenize or discover functions itself; its caller supplies those outputs.

source_statistics.cpp: lines 6-28 - Construct the record, read each original line and begin whitespace detection

```
 6 | SourceStatistics calculateSourceStatistics(
 7 |     const std::string &sourceCode,
 8 |     std::size_t tokenCount,
 9 |     std::size_t recognizedFunctionBodies)
10 | {
11 |     SourceStatistics statistics;
12 |
13 |     statistics.tokenCount = tokenCount;
14 |     statistics.recognizedFunctionBodies = recognizedFunctionBodies;
15 |
16 |     std::istringstream input(sourceCode);
17 |     std::string line;
18 |
19 |     while (std::getline(input, line))
20 |     {
21 |         ++statistics.totalLines;
22 |
23 |         bool isBlank = true;
24 |
25 |         for (char character : line)
26 |         {
27 |             if (!std::isspace(static_cast<unsigned char>(character)))
28 |             {
```

source_statistics.cpp: lines 28-41 - Mark a line as blank only if no non-whitespace character appears

```
28 |             {
29 |                 isBlank = false;
30 |                 break;
31 |             }
32 |         }
33 |
34 |         if (isBlank)
35 |         {
36 |             ++statistics.blankLines;
37 |         }
38 |     }
39 |
40 |     return statistics;
41 | }
```

accumulateSourceStatistics(aggregate, sourceStatistics)

The first parameter is a non-const reference, so the aggregate record is modified in place. Each successful call counts one file and adds that file's four measurements. The clone candidate totals are updated later by the controller, not by this helper.

source_statistics.cpp: lines 43-57 - Add one file's counters to aggregate totals

```
43 | void accumulateSourceStatistics(
44 |     AggregateStatistics &aggregate,
45 |     const SourceStatistics &sourceStatistics)
46 | {
47 |     ++aggregate.fileCount;
48 |
49 |     aggregate.totalLines += sourceStatistics.totalLines;
50 |
51 |     aggregate.blankLines += sourceStatistics.blankLines;
52 |
53 |     aggregate.totalTokens += sourceStatistics.tokenCount;
54 |
55 |     aggregate.totalRecognizedFunctionBodies +=
56 |         sourceStatistics.recognizedFunctionBodies;
57 | }
```

Walkthrough
Suppose file A has 30 lines, 5 blanks and 80 tokens; file B has 40 lines, 6 blanks and 100 tokens. Two accumulate calls produce fileCount=2, totalLines=70, blankLines=11 and totalTokens=180. Repeated uses of a token are counted in tokenCount, unlike the unique-identifier counter.

19. identifier_style: classifying unique names

enum class IdentifierStyle and the statistics record

The enum defines six mutually exclusive buckets: CamelCase, PascalCase, SnakeCase, Lowercase, SingleLetter and Other. The statistics structure tracks how many distinct names were seen, their combined character lengths, and the six bucket counts. These fields are per-file evidence; only three of the six buckets feed the historical naming model.

classifyIdentifierStyle(identifier): exact decision order

1. Empty names are Other. The first character must be alphabetic or underscore.
2. Reject any character that is neither alphanumeric nor underscore; gather flags for uppercase, lowercase and underscore usage.
3. A one-character alphabetic spelling is SingleLetter. A one-character underscore is Other.
4. If underscores exist, only qualifying lowercase-first, no-uppercase, no-double-underscore and no-trailing-underscore spellings become snake_case.
5. With no underscores: lowercase-first with uppercase somewhere becomes camelCase; uppercase-first with lowercase somewhere becomes PascalCase.
6. A lowercase-first spelling with no uppercase is lowercase; everything else is Other.

identifier_style.cpp: lines 6-19 - Initial empty-name and first-character validation

```
 6 | IdentifierStyle classifyIdentifierStyle(
 7 |     const std::string &identifier)
 8 | {
 9 |     if (identifier.empty())
10 |     {
11 |         return IdentifierStyle::Other;
12 |     }
13 |
14 |     const unsigned char first =
15 |         static_cast<unsigned char>(identifier.front());
16 |
17 |     if (!std::isalpha(first) && identifier.front() != '_')
18 |     {
19 |         return IdentifierStyle::Other;
```

identifier_style.cpp: lines 57-86 - Single-letter, snake_case, camelCase and PascalCase branches

```
57 |     if (hasUnderscore)
58 |     {
59 |         if (std::islower(first) &&
60 |             !hasUppercase &&
61 |             identifier.back() != '_' &&
62 |             identifier.find("__") == std::string::npos)
63 |         {
64 |             return IdentifierStyle::SnakeCase;
65 |         }
66 |
67 |         return IdentifierStyle::Other;
68 |     }
69 |
70 |     if (std::islower(first) && hasUppercase)
71 |     {
72 |         return IdentifierStyle::CamelCase;
73 |     }
74 |
75 |     if (std::isupper(first) && hasLowercase)
76 |     {
77 |         return IdentifierStyle::PascalCase;
78 |     }
79 |
80 |     if (std::islower(first) && !hasUppercase)
81 |     {
82 |         return IdentifierStyle::Lowercase;
83 |     }
84 |
85 |     return IdentifierStyle::Other;
86 | }
```

Identifier spelling	Why that exact category
calculateTotal	camelCase: lowercase initial, uppercase later
StudentAverage	PascalCase: uppercase initial, lowercase present
total_sum	snake_case: lowercase initial, one separator, no uppercase

Identifier spelling	Why that exact category
total	lowercase: no underscore or uppercase
x	SingleLetter: exactly one alphabetic character
_temp	Other: underscore in first position prevents qualifying snake_case
total__sum	Other: double underscore
MAX_VALUE	Other: uppercase characters with underscore

calculateIdentifierStyleStatistics(tokens)

The function visits all tokens, skips non-identifiers, inserts each identifier's lexeme into an unordered_set, and counts only the first appearance. This prevents heavily reused variable names from dominating the unique-name statistics. It adds the string length, invokes classifyIdentifierStyle, and increments one matching bucket in its switch statement.

identifier_style.cpp: lines 88-117 - Token filter, deduplication, character total and classification dispatch

```
 88 | IdentifierStyleStatistics calculateIdentifierStyleStatistics(
 89 |     const std::vector<Token> &tokens)
 90 | {
 91 |     IdentifierStyleStatistics statistics;
 92 |
 93 |     std::unordered_set<std::string> seenIdentifiers;
 94 |
 95 |     for (const Token &token : tokens)
 96 |     {
 97 |         if (token.type != TokenType::Identifier)
 98 |         {
 99 |             continue;
100 |         }
101 |
102 |         if (!seenIdentifiers.insert(token.lexeme).second)
103 |         {
104 |             continue;
105 |         }
106 |
107 |         ++statistics.uniqueIdentifierCount;
108 |
109 |         statistics.totalIdentifierCharacters +=
110 |             token.lexeme.size();
111 |
112 |         switch (classifyIdentifierStyle(token.lexeme))
113 |         {
114 |         case IdentifierStyle::CamelCase:
115 |             ++statistics.camelCase;
116 |             break;
117 |
```

identifierStyleName(style)

A final switch maps the enum back to a printable label. This keeps display names centralized so other modules can print 'camelCase' or 'snake_case' without duplicating string constants. Its fallback is Other.

identifier_style.cpp: lines 143-163 - Human-readable enum-to-label conversion

```
143 | std::string identifierStyleName(
144 |     IdentifierStyle style)
145 | {
146 |     switch (style)
147 |     {
148 |     case IdentifierStyle::CamelCase:
149 |         return "camelCase";
150 |
151 |     case IdentifierStyle::PascalCase:
152 |         return "PascalCase";
153 |
154 |     case IdentifierStyle::SnakeCase:
155 |         return "snake_case";
156 |
157 |     case IdentifierStyle::Lowercase:
158 |         return "lowercase";
159 |
160 |     case IdentifierStyle::SingleLetter:
161 |         return "single-letter";
162 |
163 |     case IdentifierStyle::Other:
```

20. identifier_history: historical naming decisions

Result fields and their meaning

Field	What it tells the caller
sufficientData	At least two earlier files and enough convention-bearing names in every sample
stableHistoricalStyle	All earlier samples have a clear and matching dominant naming style
reviewRecommended	Current sample has a clear convention different from the stable history
historicalDominantStyle	The shared old style, if established
currentDominantStyle	The current clear style, or Other when no 80% dominance

Internal helper: countConventionIdentifiers(statistics)

The sum is camelCase + PascalCase + snake_case. Lowercase, single-letter and Other counts do not contribute, because they do not show the three conventions being compared. The minimum is three distinct convention-bearing identifiers.

Internal helper: findDominantConvention(statistics)

Find the largest of those three counts, divide by their total, and require the ratio to be at least 0.80. If it is less than 0.80, return Other. The algorithm is descriptive and does not build a trained author profile.

identifier_history.cpp: lines 10-56 - Private thresholds and dominance helper

```
10 |     // Count identifiers that indicate a multiword naming convention.
11 |     std::size_t countConventionIdentifiers(
12 |         const IdentifierStyleStatistics &statistics)
13 |     {
14 |         return statistics.camelCase +
15 |                statistics.pascalCase +
16 |                statistics.snakeCase;
17 |     }
18 |
19 |     // Identify a sufficiently dominant naming convention.
20 |     IdentifierStyle findDominantConvention(
21 |         const IdentifierStyleStatistics &statistics)
22 |     {
23 |         const std::size_t total =
24 |             countConventionIdentifiers(statistics);
25 |
26 |         if (total < minimumIdentifiers)
27 |         {
28 |             return IdentifierStyle::Other;
29 |         }
30 |
31 |         IdentifierStyle dominant = IdentifierStyle::CamelCase;
32 |         std::size_t dominantCount = statistics.camelCase;
33 |
34 |         if (statistics.pascalCase > dominantCount)
35 |         {
36 |             dominant = IdentifierStyle::PascalCase;
37 |             dominantCount = statistics.pascalCase;
38 |         }
39 |
40 |         if (statistics.snakeCase > dominantCount)
41 |         {
42 |             dominant = IdentifierStyle::SnakeCase;
43 |             dominantCount = statistics.snakeCase;
44 |         }
45 |
46 |         const double proportion =
47 |             static_cast<double>(dominantCount) /
48 |             static_cast<double>(total);
49 |
50 |         if (proportion < dominanceThreshold)
51 |         {
52 |             return IdentifierStyle::Other;
53 |         }
54 |
55 |         return dominant;
56 |     }
```

Public function: analyzeIdentifierHistory(historicalFiles, currentFile)

1. Begin with flags false and the style enums set to Other.
2. Require historicalFiles.size() >= 2, then test the minimum name count of each historical and current sample.
3. Mark sufficientData=true once these counts pass.
4. Evaluate each old assignment's dominant convention. Any unclear or conflicting old file ends the check without a stable baseline.
5. When all old conventions agree, set stableHistoricalStyle and record the historical style.
6. Calculate the current dominant convention. Set reviewRecommended only if it is clear and different from the historical style.

identifier_history.cpp: lines 59-91 - Minimum evidence gates before historical comparison

```
59 | IdentifierHistoryResult analyzeIdentifierHistory(
60 |     const std::vector<IdentifierStyleStatistics> &historicalFiles,
61 |     const IdentifierStyleStatistics &currentFile)
62 | {
63 |     IdentifierHistoryResult result;
64 |
65 |     // Require at least two previous submissions.
66 |     if (historicalFiles.size() < 2)
67 |     {
68 |         return result;
69 |     }
70 |
71 |     // Ensure every historical file has enough relevant identifiers.
72 |     for (const IdentifierStyleStatistics &statistics : historicalFiles)
73 |     {
74 |         if (countConventionIdentifiers(statistics) < minimumIdentifiers)
75 |         {
76 |             return result;
77 |         }
78 |     }
79 |
80 |     // Ensure the current submission also has enough identifiers.
81 |     if (countConventionIdentifiers(currentFile) < minimumIdentifiers)
82 |     {
83 |         return result;
84 |     }
85 |
86 |     result.sufficientData = true;
87 |
88 |     IdentifierStyle historicalStyle = IdentifierStyle::Other;
89 |
90 |     // Examine the naming convention of each historical submission.
91 |     for (const IdentifierStyleStatistics &statistics : historicalFiles)
```

identifier_history.cpp: lines 109-125 - Stable baseline, current style and final review condition

```
109 |     }
110 |
111 |     result.stableHistoricalStyle = true;
112 |     result.historicalDominantStyle = historicalStyle;
113 |
114 |     // Determine the current submission's dominant convention.
115 |     result.currentDominantStyle =
116 |         findDominantConvention(currentFile);
117 |
118 |     // Recommend review only when a clear convention change exists.
119 |     if (result.currentDominantStyle != IdentifierStyle::Other &&
120 |         result.currentDominantStyle != historicalStyle)
121 |     {
122 |         result.reviewRecommended = true;
123 |     }
124 |
125 |     return result;
```

Worked example

Sample	camelCase	PascalCase	snake_case	Classification
assignment1	5	0	0	camelCase, 100%
assignment2	4	0	1	camelCase, 80%
assignment3 (current)	0	0	5	snake_case, 100%

Both earlier assignments meet the 80% threshold and agree. The current style is clearly snake_case, so the function returns sufficientData=true, stableHistoricalStyle=true, reviewRecommended=true. If the old files disagreed, the answer would be not evaluable, not 'no change'.

21. comment_style: restoring evidence lost by cleanup

Why this module is called before tokenization

The preprocessor intentionally blanks supported comments, preserving line endings. Once only cleaned text remains, the original comment content is gone. application.cpp therefore retains both versions and invokes calculateCommentStyleStatistics(original, cleaned) immediately after removeComments().

calculateCommentStyleStatistics(sourceCode, cleanedCode)

1. Construct two istringstream objects, one for original and one for cleaned source.
2. Read corresponding physical lines with getline in the same loop.
3. If a pair is identical, there was no recognized comment removal on that line, so continue.

4. Otherwise increment commentLines. Test cleanedLine using std::any_of to find any non-whitespace character.
5. If executable/noncomment text remains, increment inlineCommentLines; otherwise increment commentOnlyLines.

comment_style.cpp: lines 7-45 - Original-versus-cleaned line comparison and std::any_of lambda

```
 7 | CommentStyleStatistics calculateCommentStyleStatistics(
 8 |     const std::string &sourceCode,
 9 |     const std::string &cleanedCode)
10 | {
11 |     CommentStyleStatistics statistics;
12 |
13 |     std::istringstream originalInput(sourceCode);
14 |     std::istringstream cleanedInput(cleanedCode);
15 |
16 |     std::string originalLine;
17 |     std::string cleanedLine;
18 |
19 |     // Compare the original and comment-removed source line by line.
20 |     while (std::getline(originalInput, originalLine) &&
21 |            std::getline(cleanedInput, cleanedLine))
22 |     {
23 |         // An unchanged line contains no recognized comments.
24 |         if (originalLine == cleanedLine)
25 |         {
26 |             continue;
27 |         }
28 |
29 |         ++statistics.commentLines;
30 |
31 |         // Check whether executable code remains after comment removal.
32 |         const bool containsCode = std::any_of(
33 |             cleanedLine.begin(),
34 |             cleanedLine.end(),
35 |             [](unsigned char character)
36 |             {
37 |                 return !std::isspace(character);
38 |             });
39 |
40 |         if (containsCode)
41 |         {
42 |             ++statistics.inlineCommentLines;
43 |         }
44 |         else
45 |         {
```

Original line	Cleaned version	Class
// file header	blank spaces	Comment-only
int x=2; // assign	int x=2; plus blanks	Inline-comment
int y=1;	int y=1;	No recognized comment
const char* s="//";	Same literal text	No recognized comment

Original line	Cleaned version	Class
int a=1; /* note */	int a=1; plus blanks	Inline-comment

The feature's tests cover // comments, /* */ blocks, multiline blocks, URLs/comment markers inside ordinary string literals, empty source, and mixed line/inline comments. It does not parse every advanced C++ literal form, so the classification is bounded by the supported preprocessing scanner.

22. comment_history: rate calculation and safeguards

CommentHistorySample and CommentHistoryResult

Each sample stores totalLines, blankLines and commentLines. The result has sufficientData, stableHistoricalRate and reviewRecommended flags, plus the historical and current floating-point rates. It does not save every original comment or infer why a student changed their commenting habits.

Private helper: isUsableSample(sample)

Reject blankLines greater than totalLines. Then compute nonblankLines = totalLines - blankLines. Require nonblankLines >= 20 and commentLines <= nonblankLines. These checks avoid nonsensical inputs and tiny samples.

comment_history.cpp: lines 6-35 - Three constants, sample validation and double-precision comment rate

```
 6 | namespace
 7 | {
 8 |     constexpr std::size_t minimumNonblankLines = 20;
 9 |     constexpr double historyTolerance = 0.10;
10 |     constexpr double deviationThreshold = 0.20;
11 |
12 |     bool isUsableSample(const CommentHistorySample &sample)
13 |     {
14 |         if (sample.blankLines > sample.totalLines)
15 |         {
16 |             return false;
17 |         }
18 |
19 |         const std::size_t nonblankLines =
20 |             sample.totalLines - sample.blankLines;
21 |
22 |         return nonblankLines >= minimumNonblankLines &&
23 |                sample.commentLines <= nonblankLines;
24 |     }
25 |
26 |     double calculateCommentRate(
27 |         const CommentHistorySample &sample)
28 |     {
29 |         const std::size_t nonblankLines =
30 |             sample.totalLines - sample.blankLines;
31 |
32 |         return static_cast<double>(sample.commentLines) /
33 |                static_cast<double>(nonblankLines);
34 |     }
35 | }
```

Public function: analyzeCommentHistory(historicalFiles, currentFile)

1. Require two earlier samples and a valid current sample. Reject invalid earlier samples.
2. Accumulate total historical comment lines and nonblank lines, while tracking minimum and maximum old per-file rates.
3. Set sufficientData=true; compute pooled history rate as total historical comments / total historical nonblank lines.
4. Require maxHistoricalRate - minHistoricalRate <= 0.10. Otherwise the old commenting behavior is not stable.
5. When stable, compare fabs(currentRate - historicalRate) with 0.20. A difference of 20 percentage points or more sets reviewRecommended=true.

comment_history.cpp: lines 37-65 - Initial validation and historical totals

```
37 | CommentHistoryResult analyzeCommentHistory(
38 |     const std::vector<CommentHistorySample> &historicalFiles,
39 |     const CommentHistorySample &currentFile)
40 | {
41 |     CommentHistoryResult result;
42 |
43 |     if (historicalFiles.size() < 2 ||
44 |         !isUsableSample(currentFile))
45 |     {
46 |         return result;
47 |     }
48 |
49 |     std::size_t totalHistoricalComments = 0;
50 |     std::size_t totalHistoricalNonblankLines = 0;
51 |
52 |     double minimumRate = 1.0;
53 |     double maximumRate = 0.0;
54 |
55 |     for (const CommentHistorySample &file : historicalFiles)
56 |     {
57 |         if (!isUsableSample(file))
58 |         {
59 |             return result;
60 |         }
61 |
62 |         const std::size_t nonblankLines =
63 |             file.totalLines - file.blankLines;
64 |
65 |         totalHistoricalComments += file.commentLines;
```

comment_history.cpp: lines 76-102 - Stability test and absolute rate change

```
 76 |     result.historicalCommentRate =
 77 |         static_cast<double>(totalHistoricalComments) /
 78 |         static_cast<double>(totalHistoricalNonblankLines);
 79 |
 80 |     result.currentCommentRate =
 81 |         calculateCommentRate(currentFile);
 82 |
 83 |     // Historical submissions must show a reasonably consistent rate.
 84 |     if (maximumRate - minimumRate > historyTolerance)
 85 |     {
 86 |         return result;
 87 |     }
 88 |
 89 |     result.stableHistoricalRate = true;
 90 |
 91 |     // Detect a substantial change in the current submission.
 92 |     const double difference = std::fabs(
 93 |         result.currentCommentRate -
 94 |         result.historicalCommentRate);
 95 |
 96 |     if (difference >= deviationThreshold)
 97 |     {
 98 |         result.reviewRecommended = true;
 99 |     }
100 |
101 |     return result;
102 | }
```

Worked numeric example with pooled rates

Suppose assignment1 has 25 nonblank lines and 8 commented lines (8/25 = 32%). Assignment2 has 25 nonblank lines and 9 commented lines (9/25 = 36%). The old spread is just 4 percentage points, below the 10-point stability allowance. Pooled history is (8+9)/(25+25) = 34%. If the current file has 0 commented lines out of 28 nonblank lines, the difference is 34 percentage points, triggering review.

If old rates were 0% and 40%, the history would be unstable; the function would refuse to classify a current change even if the current rate were 0%. Display formatting rounds with std::lround(rate*100), but decisions use the unrounded double values.

23. assignment_order: reliable current-vs-past ordering

Why file-system listing order is not a timeline

On many systems, lexical filename order places assignment10.cpp before assignment2.cpp. The module extracts an unsigned numeric assignment suffix, validates the entire filename, and uses std::map to order the files by integer value. It returns original positions, allowing application.cpp to reorder the already-analyzed file indexes without losing their measurements.

Private function: parseAssignmentNumber(filename, number)

1. Find the last dot. Require exactly the .c or .cpp extension.
2. Remove the extension and verify the 'assignment' prefix without case sensitivity.
3. Skip at most one optional underscore or hyphen separator after the prefix.
4. Require at least one digit and reject any unexpected character.
5. Before multiplying accumulated number by 10 and adding digit, use the mathematical overflow guard.
6. Write the parsed std::size_t number to the output reference and return true.

assignment_order.cpp: lines 9-26 - Filename extension check and stem extraction

```
 9 |     bool parseAssignmentNumber(
10 |         const std::string &filename,
11 |         std::size_t &number)
12 |     {
13 |         const std::size_t dot = filename.find_last_of('.');
14 |
15 |         if (dot == std::string::npos)
16 |         {
17 |             return false;
18 |         }
19 |
20 |         const std::string extension = filename.substr(dot);
21 |
22 |         if (extension != ".c" && extension != ".cpp")
23 |         {
24 |             return false;
25 |         }
26 |
```

assignment_order.cpp: lines 47-88 - Optional separator, digit parsing and overflow rejection

```
47 |         std::size_t position = prefix.size();
48 |
49 |         // Allow assignment_3 and assignment-3.
50 |         if (stem[position] == '_' || stem[position] == '-')
51 |         {
52 |             ++position;
53 |         }
54 |
55 |         if (position == stem.size())
56 |         {
57 |             return false;
58 |         }
59 |
60 |         std::size_t value = 0;
61 |
62 |         // Read the number while checking for integer overflow.
63 |         for (; position < stem.size(); ++position)
64 |         {
65 |             const unsigned char character =
66 |                 static_cast<unsigned char>(stem[position]);
67 |
68 |             if (!std::isdigit(character))
69 |             {
70 |                 return false;
71 |             }
72 |
73 |             const std::size_t digit =
74 |                 static_cast<std::size_t>(stem[position] - '0');
75 |
76 |             const std::size_t maximum =
77 |                 std::numeric_limits<std::size_t>::max();
78 |
79 |             if (value > (maximum - digit) / 10)
80 |             {
81 |                 return false;
82 |             }
83 |
84 |             value = value * 10 + digit;
85 |         }
86 |
87 |         number = value;
88 |         return true;
```

The crucial overflow guard is value > (maximum - digit)/10. If this expression is true, then value*10+digit would exceed std::size_t's maximum; returning false prevents overflow before it happens.

Public function: determineAssignmentOrder(filenames)

The function builds a std::map<std::size_t,std::size_t> of assignmentNumber to original list position. std::map sorts its numeric keys; emplace(...).second detects duplicate numbers, including assignment01.cpp versus assignment1.c. Invalid input sets valid=false and a human-readable reason. Valid input yields the orderedPositions vector and valid=true.

assignment_order.cpp: lines 92-138 - Full order-validation and map-to-position logic

```
 92 | AssignmentOrderResult determineAssignmentOrder(
 93 |     const std::vector<std::string> &filenames)
 94 | {
 95 |     AssignmentOrderResult result;
 96 |
 97 |     if (filenames.empty())
 98 |     {
 99 |         result.reason = "No assignment files provided.";
100 |         return result;
101 |     }
102 |
103 |     // A map automatically orders numeric assignment keys.
104 |     std::map<std::size_t, std::size_t> numberedFiles;
105 |
106 |     for (std::size_t i = 0; i < filenames.size(); ++i)
107 |     {
108 |         std::size_t assignmentNumber = 0;
109 |
110 |         if (!parseAssignmentNumber(
111 |                 filenames[i], assignmentNumber))
112 |         {
113 |             result.reason =
114 |                 "Unsupported assignment filename: " + filenames[i];
115 |             return result;
116 |         }
117 |
118 |         const bool inserted =
119 |             numberedFiles.emplace(assignmentNumber, i).second;
120 |
121 |         if (!inserted)
122 |         {
123 |             result.reason =
124 |                 "Duplicate assignment number: " +
125 |                 std::to_string(assignmentNumber);
126 |
127 |             return result;
128 |         }
129 |     }
130 |
131 |     for (const auto &entry : numberedFiles)
132 |     {
133 |         result.orderedPositions.push_back(entry.second);
134 |     }
135 |
136 |     result.valid = true;
137 |     return result;
138 | }
```

Discovery positions	Parsed numbers	Returned positions
0: assignment10.cpp; 1: assignment2.cpp; 2: assignment1.cpp	10, 2, 1	2, 1, 0

Important distinction
The historical analysis is skipped when the student's assignment filenames are ambiguous, but the successfully opened files remain eligible for ordinary Type-1/Type-2 body comparisons. The module prevents a false historical baseline; it does not discard the submissions.

24. clone_summary: exact evidence with both source locations

The two public types

CloneCandidateKind identifies Type1 or Type2Only. CloneSummaryEntry stores kind, firstPath, firstStartLine, firstEndLine, secondPath, secondStartLine and secondEndLine. A single entry is therefore one specific body-level match between two source locations, not merely a student-level label.

Private helper: validRegion(tokens, region)

A valid body must have beginTokenIndex less than endTokenIndex, and endTokenIndex no larger than tokens.size(). This prevents an empty interval and out-of-bounds vector access. Because endTokenIndex is exclusive, the last token lies at endTokenIndex - 1.

clone_summary.cpp: lines 10-15 - Check a supported nonempty token-index interval

```
10 |     bool validRegion(const std::vector<Token> &tokens,
11 |                      const CodeRegion &region)
12 |     {
13 |         return region.beginTokenIndex < region.endTokenIndex &&
14 |                region.endTokenIndex <= tokens.size();
15 |     }
```

Private helper: studentFolderFromPath(relativePath)

Convert the relative string to std::filesystem::path. If it has no parent path, return an empty student ID. Otherwise take its first path component as the student folder, matching application.cpp's grouping rule. This works for student01/assignment3.cpp while leaving a root-level file unassigned.

clone_summary.cpp: lines 17-27 - Extract student ID from the first relative folder

```
17 |     std::string studentFolderFromPath(const std::string &relativePath)
18 |     {
19 |         const std::filesystem::path path(relativePath);
20 |
21 |         if (!path.has_parent_path())
22 |         {
23 |             return "";
24 |         }
25 |
26 |         return (*path.begin()).string();
27 |     }
```

Private helper: printLocation(output, path, startLine, endLine)

Choose the output label: Student + Assignment for an assignment-named file, Student + File for another filename inside a student folder, or File for a root-level path. It prints both start and end inclusive source lines from the stored record.

clone_summary.cpp: lines 29-61 - Render one source location with student and filename labels

```
29 |     void printLocation(std::ostream &output,
30 |                        const std::string &relativePath,
31 |                        std::size_t startLine,
32 |                        std::size_t endLine)
33 |     {
34 |         const std::string student = studentFolderFromPath(relativePath);
35 |         const std::string filename =
36 |             std::filesystem::path(relativePath).filename().string();
37 |
38 |         output << "    ";
39 |
40 |         if (!student.empty())
41 |         {
42 |             output << "Student: " << student << " | ";
43 |
44 |             if (filename.rfind("assignment", 0) == 0)
45 |             {
46 |                 output << "Assignment: " << filename;
47 |             }
48 |             else
49 |             {
50 |                 output << "File: " << filename;
51 |             }
52 |         }
53 |         else
54 |         {
55 |             output << "File: " << relativePath;
56 |         }
57 |
58 |         output << " | lines " << startLine
59 |                << " to " << endLine << '\n';
60 |     }
61 | }
```

appendCloneSummaryEntry(entries, kind, first..., second...)

If either region is invalid, return false and add nothing. Otherwise extract first and last token line numbers from both regions, construct one CloneSummaryEntry, then compare secondPath and firstPath. If secondPath sorts first, swap both file paths and both associated line-boundary pairs together. Append the canonical record and return true.

clone_summary.cpp: lines 63-83 - Guard against invalid inputs and build the entry

```
63 | bool appendCloneSummaryEntry(
64 |     std::vector<CloneSummaryEntry> &entries,
65 |     CloneCandidateKind kind,
66 |     const std::string &firstRelativePath,
67 |     const std::vector<Token> &firstTokens,
68 |     const CodeRegion &firstRegion,
69 |     const std::string &secondRelativePath,
70 |     const std::vector<Token> &secondTokens,
71 |     const CodeRegion &secondRegion)
72 | {
73 |     if (!validRegion(firstTokens, firstRegion) ||
74 |         !validRegion(secondTokens, secondRegion))
75 |     {
76 |         return false;
77 |     }
78 |
79 |     CloneSummaryEntry entry{
80 |         kind,
81 |         firstRelativePath,
82 |         firstTokens[firstRegion.beginTokenIndex].line,
83 |         firstTokens[firstRegion.endTokenIndex - 1].line,
```

clone_summary.cpp: lines 84-99 - Canonicalize file order without mixing up the line numbers

```
84 |         secondRelativePath,
85 |         secondTokens[secondRegion.beginTokenIndex].line,
86 |         secondTokens[secondRegion.endTokenIndex - 1].line};
87 |
88 |     // Canonicalize file order so a pair has one predictable presentation.
89 |     if (entry.secondPath < entry.firstPath)
90 |     {
91 |         std::swap(entry.firstPath, entry.secondPath);
92 |         std::swap(entry.firstStartLine, entry.secondStartLine);
93 |         std::swap(entry.firstEndLine, entry.secondEndLine);
94 |     }
95 |
96 |     entries.push_back(entry);
97 |
98 |     return true;
99 | }
```

printCloneSummaryDetails(output, entries)

If empty, print an explanatory no-candidate message. Otherwise make a copy and sort it with std::sort and std::tie over clone kind, paths and line numbers. Iterate first through Type-1 candidates, then Type-2-only candidates. Each group has its own Candidate #1, #2, ... counter. This ensures repeatable report order even when filesystem traversal order varies.

clone_summary.cpp: lines 101-134 - Handle the empty case and sort candidates deterministically

```
101 | void printCloneSummaryDetails(
102 |     std::ostream &output,
103 |     const std::vector<CloneSummaryEntry> &entries)
104 | {
105 |     output << "\nDetailed clone candidate summary:\n";
106 |
107 |     if (entries.empty())
108 |     {
109 |         output << "No body-level clone candidates found.\n";
110 |         return;
111 |     }
112 |
113 |     std::vector<CloneSummaryEntry> sorted = entries;
114 |
115 |     std::sort(sorted.begin(), sorted.end(),
116 |               [](const CloneSummaryEntry &left,
117 |                  const CloneSummaryEntry &right)
118 |               {
119 |                   return std::tie(left.kind, left.firstPath,
120 |                                   left.secondPath, left.firstStartLine,
121 |                                   left.secondStartLine) <
122 |                          std::tie(right.kind, right.firstPath,
123 |                                   right.secondPath, right.firstStartLine,
124 |                                   right.secondStartLine);
125 |               });
126 |
127 |     for (const CloneCandidateKind kind :
128 |          {CloneCandidateKind::Type1, CloneCandidateKind::Type2Only})
129 |     {
130 |         output << '\n'
131 |                << (kind == CloneCandidateKind::Type1
132 |                        ? "Type-1 candidate pairs:\n"
133 |                        : "Type-2-only candidate pairs:\n");
134 |
```

Stored location A	Stored location B	Rendered evidence
student05/assignment3.cpp, 34-64	student12/assignment3.cpp, 32-62	Type-2-only candidate with both independent boundaries
student19/assignment3.cpp, 4-29	student03/assignment3.cpp, 4-29	Printed as student03 first after canonicalization

25. analysis_report: combining detection and style evidence

StudentStyleFinding: one record per grouped student

Field	Meaning
studentId	First relative subfolder name such as student05
namingEvaluated	True only when sufficient identifier evidence and stable naming history exist
namingDeviation	True only when evaluated and a clear naming change is detected
commentingEvaluated	True only when sufficient source lines and stable old comment rate exist
commentingDeviation	True only when evaluated and a threshold-level rate change is detected

AnalysisReport: the transport record

AnalysisReport bundles studentFolderCount, comparedFilePairs, AggregateStatistics, vector<CloneSummaryEntry> and vector<StudentStyleFinding>. It is a data-transfer record: source detection has already happened. The printer should not rerun tokenization or clone detection. This makes the planned JSON/CSV exporter a clean future consumer of the same evidence.

Private helper: studentFromPath(relativePath)

As in clone_summary, return the first path component for a nested source path. If no student directory exists, return an empty string, preventing fabricated student IDs.

analysis_report.cpp: lines 9-20 - Recover the student folder from a match path

```
 9 |     std::string studentFromPath(const std::string &relativePath)
10 |     {
11 |         const std::filesystem::path path(relativePath);
12 |
13 |         if (!path.has_parent_path())
14 |         {
15 |             return "";
16 |         }
17 |
18 |         // Match application.cpp: the first relative folder is the student ID.
19 |         return (*path.begin()).string();
20 |     }
```

Private helper: printStudentList(output, set)

A std::set contains unique, sorted student IDs. Print None when empty; otherwise print comma-separated names with a boolean 'first' separator check. This avoids repeated student names when one student appears in many candidate pairs.

analysis_report.cpp: lines 22-42 - Print one stable, deduplicated list

```
22 |     void printStudentList(std::ostream &output,
23 |                           const std::set<std::string> &students)
24 |     {
25 |         if (students.empty())
26 |         {
27 |             output << "None";
28 |             return;
29 |         }
30 |
31 |         bool first = true;
32 |         for (const std::string &student : students)
33 |         {
34 |             if (!first)
35 |             {
36 |                 output << ", ";
37 |             }
38 |             output << student;
39 |             first = false;
40 |         }
41 |     }
42 | }
```

printCombinedAnalysisReport(output, report)

1. Start empty sets for Type-1 participants, Type-2-only participants, naming/commenting changes, and not-evaluable histories.
2. Traverse each CloneSummaryEntry. Extract both student folders, insert them into the corresponding category set, and insert one reason into reviewReasons[student].
3. Traverse each StudentStyleFinding. Count evaluable histories, distinguish deviations from unevaluable cases, and add qualifying reasons to the same reviewReasons map.
4. Print six report sections: dataset overview; statistics; clone candidates with full paths and lines; stylometry; students needing human review; final ethical statement.
5. The number of students requiring review is reviewReasons.size(), not the count of clone pairs or style rules.

analysis_report.cpp: lines 44-77 - Build participant sets and reason map from real candidate entries

```
44 | void printCombinedAnalysisReport(
45 |     std::ostream &output,
46 |     const AnalysisReport &report)
47 | {
48 |     std::set<std::string> type1Students;
49 |     std::set<std::string> type2Students;
50 |     std::map<std::string, std::set<std::string>> reviewReasons;
51 |
52 |     for (const CloneSummaryEntry &candidate : report.cloneCandidates)
53 |     {
54 |         const std::string first = studentFromPath(candidate.firstPath);
55 |         const std::string second = studentFromPath(candidate.secondPath);
56 |         std::set<std::string> &participants =
57 |             candidate.kind == CloneCandidateKind::Type1
58 |                 ? type1Students
59 |                 : type2Students;
60 |         const std::string reason =
61 |             candidate.kind == CloneCandidateKind::Type1
62 |                 ? "Type-1 body-level candidate"
63 |                 : "Type-2-only body-level candidate";
64 |
65 |         for (const std::string &student : {first, second})
66 |         {
67 |             if (!student.empty())
68 |             {
69 |                 participants.insert(student);
70 |                 reviewReasons[student].insert(reason);
71 |             }
72 |         }
73 |     }
74 |
75 |     std::size_t namingCount = 0;
76 |     std::size_t commentingCount = 0;
77 |     std::size_t namingEvaluable = 0;
```

analysis_report.cpp: lines 84-116 - Evaluate naming and commenting flags separately

```
    for (const StudentStyleFinding &finding : report.studentFindings)
    {
        if (finding.namingEvaluated)
        {
            ++namingEvaluable;
            if (finding.namingDeviation)
            {
                ++namingCount;
                namingStudents.insert(finding.studentId);
                reviewReasons[finding.studentId].insert(
                    "Historical naming-style deviation");
            }
        }
        else
        {
            namingNotEvaluable.insert(finding.studentId);
        }

        if (finding.commentingEvaluated)
        {
            ++commentingEvaluable;
            if (finding.commentingDeviation)
            {
                ++commentingCount;
                commentingStudents.insert(finding.studentId);
                reviewReasons[finding.studentId].insert(
                    "Historical commenting-style deviation");
            }
        }
        else
        {
            commentingNotEvaluable.insert(finding.studentId);
        }
```

analysis_report.cpp: lines 174-203 - Print the deduplicated student reasons and the final caution

```
174 |     output << '\n';
175 |
176 |     output << "\n5. HUMAN REVIEW INDICATORS\n"
177 |            << "-------------------------------------------------\n"
178 |            << "Students with at least one reported indicator: "
179 |            << reviewReasons.size() << '\n';
180 |
181 |     if (reviewReasons.empty())
182 |     {
183 |         output << "No clone or stable-history deviation indicators found.\n";
184 |     }
185 |     for (const auto &student : reviewReasons)
186 |     {
187 |         output << "Student: " << student.first << '\n';
188 |         for (const std::string &reason : student.second)
189 |         {
190 |             output << "- " << reason << '\n';
191 |         }
192 |     }
193 |
194 |     output << "\n6. FINAL REPORT\n"
195 |            << "-------------------------------------------------\n"
196 |            << "Detailed candidate locations: Included above\n"
197 |            << "Statistical findings: Included above\n"
198 |            << "Important: Similarity and coding-style changes are indicators\n"
199 |            << "for human review, not proof of plagiarism.\n"
200 |            << "=================================================\n"
201 |            << "                 ANALYSIS COMPLETE\n"
202 |            << "=================================================\n";
203 | }
```

Why map<string, set<string>> matters

Consider student05. The report may contain four different Fibonacci match pairs involving student05, plus a naming deviation and a commenting deviation. Without sets, student05 might appear repeatedly and its reasons might be duplicated. Using std::map<student, std::set<reason>> produces one student entry with the three distinct reasons. That is why 20 reported pair matches can still lead to only 12 unique students under review.

26. application.cpp: the complete integration controller

Entry point and the new per-file object

main.cpp delegates to runCloneScope(). Near the top of application.cpp, AnalyzedSourceFile stores a filesystem path, vector of Token, vector of CodeRegion, SourceStatistics, IdentifierStyleStatistics and CommentStyleStatistics. This struct is the memory record reused by history analysis and pair comparisons, so the application need not reopen or retokenize files for each comparison.

application.cpp: lines 56-66 - The per-file record holds every downstream prerequisite

```
56 | struct AnalyzedSourceFile
57 | {
58 |     fs::path path;
59 |     std::vector<Token> tokens;
60 |     std::vector<CodeRegion> functionRegions;
61 |     SourceStatistics statistics;
62 |     IdentifierStyleStatistics identifierStats;
63 |     CommentStyleStatistics commentStats;
64 | };
65 |
66 | int runCloneScope()
```

Directory input: file discovery

1. Ask for one path using getline(cin, filePath) and construct fs::path sourcePath.
2. If fs::is_directory(sourcePath) is true, ask whether to recurse. 'y' or 'Y' selects recursive_directory_iterator; other answers select directory_iterator.
3. Collect directory_entry objects, then process regular files whose extensions are lowercase .c or .cpp.
4. Open each file with ifstream. If opening fails, print an error and skip it; if it succeeds, read its full contents via ostringstream and rdbuf().

application.cpp: lines 76-115 - Input selection and recursive/non-recursive source discovery

```
 76 |     string filePath;
 77 |
 78 |     cout << "Enter source file path: ";
 79 |     getline(cin, filePath);
 80 |
 81 |     fs::path sourcePath(filePath);
 82 |
 83 |     // Check whether the input path is a directory
 84 |     if (fs::is_directory(sourcePath))
 85 |     {
 86 |         cout << "Folder detected successfully!\n";
 87 |
 88 |         std::vector<AnalyzedSourceFile> analyzedFiles;
 89 |
 90 |         AggregateStatistics aggregate;
 91 |
 92 |         std::vector<fs::directory_entry> entries;
 93 |
 94 |         string scanChoice;
 95 |
 96 |         cout << "Scan subfolders recursively? (y/n): ";
 97 |         getline(cin, scanChoice);
 98 |         if (scanChoice == "y" || scanChoice == "Y")
 99 |         {
100 |             for (const auto &entry :
101 |                  fs::recursive_directory_iterator(sourcePath))
102 |             {
103 |                 entries.push_back(entry);
104 |             }
105 |         }
106 |         else
107 |         {
108 |             for (const auto &entry :
109 |                  fs::directory_iterator(sourcePath))
110 |             {
111 |                 entries.push_back(entry);
112 |             }
113 |         }
114 |
115 |         // Visit each entry directly inside the folder
```

Per-file calculation in the right dependency order

1. Store original sourceCode; call removeComments(sourceCode) to obtain cleanedCode.
2. Immediately call calculateCommentStyleStatistics(sourceCode, cleanedCode) while both versions remain available.
3. Call tokenize(cleanedCode), then findFunctionBodyRegions(tokens).
4. Call calculateSourceStatistics(original sourceCode, tokens.size(), region count).
5. Call calculateIdentifierStyleStatistics(tokens); print per-file metrics.
6. Call accumulateSourceStatistics(aggregate, statistics) and push the six-field AnalyzedSourceFile record into analyzedFiles.

application.cpp: lines 139-169 - The exact preprocessing, commenting, tokenizer and source-statistics call order

```
139 |                 std::string sourceCode = buffer.str();
140 |
141 |                 // Remove comments from this source file
142 |                 std::string cleanedCode = removeComments(sourceCode);
143 |
144 |                 // Analyze comments using the original and cleaned source.
145 |                 CommentStyleStatistics commentStats =
146 |                     calculateCommentStyleStatistics(sourceCode, cleanedCode);
147 |
148 |                 // Display the processed source code
149 |                 cout << "Preprocessed source code:\n";
150 |                 cout << cleanedCode << '\n';
151 |
152 |                 // Tokenize the current source file.
153 |                 std::vector<Token> tokens = tokenize(cleanedCode);
154 |
155 |                 // Display the total token count for this file.
156 |                 cout << "Total tokens: " << tokens.size() << '\n';
157 |
158 |                 std::vector<CodeRegion> functionRegions =
159 |                     findFunctionBodyRegions(tokens);
160 |
161 |                 cout << "Supported function-body regions: "
162 |                      << functionRegions.size() << '\n';
163 |
164 |                 // Calculate statistics for the current source file.
165 |                 SourceStatistics statistics = calculateSourceStatistics(
166 |                     sourceCode,
167 |                     tokens.size(),
168 |                     functionRegions.size());
169 |
```

application.cpp: lines 188-223 - Identifier feature calculation and storing the analyzed file

```
188 |                     calculateIdentifierStyleStatistics(tokens);
189 |
190 |                 // Display the number of unique identifiers.
191 |                 cout << "Unique identifier names: "
192 |                      << identifierStats.uniqueIdentifierCount << '\n';
193 |
194 |                 // Display the naming-style distribution.
195 |                 cout << "camelCase identifiers: "
196 |                      << identifierStats.camelCase << '\n';
197 |
198 |                 cout << "PascalCase identifiers: "
199 |                      << identifierStats.pascalCase << '\n';
200 |
201 |                 cout << "snake_case identifiers: "
202 |                      << identifierStats.snakeCase << '\n';
203 |
204 |                 cout << "Lowercase identifiers: "
205 |                      << identifierStats.lowercase << '\n';
206 |
207 |                 cout << "Single-letter identifiers: "
208 |                      << identifierStats.singleLetter << '\n';
209 |
210 |                 cout << "Other identifier styles: "
211 |                      << identifierStats.other << '\n';
212 |
213 |                 // Add this file's statistics to the aggregate totals.
214 |                 accumulateSourceStatistics(aggregate, statistics);
215 |
216 |                 analyzedFiles.push_back(
217 |                     AnalyzedSourceFile{
218 |                         entry.path(),
219 |                         tokens,
220 |                         functionRegions,
221 |                         statistics,
222 |                         identifierStats,
223 |                         commentStats});
```

Student grouping and chronology

The controller allocates std::map<std::string, std::vector<std::size_t>> studentFileIndices. It calculates each path relative to the user's selected root and uses the first folder component as student ID. Files placed directly in the selected root have no student folder and do not enter a student history group, but still enter all-file clone comparisons.

application.cpp: lines 230-252 - Student ID extraction and group accumulation

```
230 |         // Group analyzed source files by their student folder.
231 |         std::map<std::string, std::vector<std::size_t>> studentFileIndices;
232 |
233 |         for (std::size_t i = 0; i < analyzedFiles.size(); ++i)
234 |         {
235 |             const fs::path relativePath =
236 |                 analyzedFiles[i].path.lexically_relative(sourcePath);
237 |
238 |             // Files directly inside the selected folder have no student folder.
239 |             if (!relativePath.has_parent_path())
240 |             {
241 |                 continue;
242 |             }
243 |
244 |             // The first folder component represents the student.
245 |             const std::string studentId =
246 |                 (*relativePath.begin()).string();
247 |
248 |             studentFileIndices[studentId].push_back(i);
249 |         }
250 |
251 |         // Display the detected student groups.
252 |         cout << "\nStudent folders detected: "
```

For each grouped student, it requires at least three files, collects their filenames, calls determineAssignmentOrder, then remaps the sorted positions back to analyzedFiles indexes. It builds historical vectors from all but the last numbered assignment, selects the last as current, and calls both historical analyzers independently.

application.cpp: lines 274-307 - Evidence gate and assignment chronology validation

```
274 |
275 |             // Require at least two historical files and one current file.
276 |             if (fileIndices.size() < 3)
277 |             {
278 |                 cout << "Student: " << student.first
279 |                      << " | Insufficient historical submissions.\n";
280 |                 continue;
281 |             }
282 |
283 |             // Collect filenames for numeric assignment ordering.
284 |             std::vector<std::string> filenames;
285 |
286 |             for (std::size_t index : fileIndices)
287 |             {
288 |                 filenames.push_back(
289 |                     analyzedFiles[index].path.filename().string());
290 |             }
291 |
292 |             // Validate filenames and determine chronological order.
293 |             const AssignmentOrderResult assignmentOrder =
294 |                 determineAssignmentOrder(filenames);
295 |
296 |             // Avoid historical comparisons when assignment order is uncertain.
297 |             if (!assignmentOrder.valid)
298 |             {
299 |                 cout << "Student: " << student.first
300 |                      << " | Historical analysis skipped: "
301 |                      << assignmentOrder.reason << '\n';
302 |
303 |                 continue;
304 |             }
305 |
306 |             // Translate the ordered positions back to analyzed-file indices.
307 |             std::vector<std::size_t> orderedFileIndices;
```

application.cpp: lines 320-354 - Build historical samples and call both style analyzers

```
320 |             for (std::size_t i = 0; i + 1 < fileIndices.size(); ++i)
321 |             {
322 |                 const AnalyzedSourceFile &historicalFile =
323 |                     analyzedFiles[fileIndices[i]];
324 |
325 |                 historicalFiles.push_back(
326 |                     historicalFile.identifierStats);
327 |
328 |                 historicalCommentFiles.push_back(
329 |                     CommentHistorySample{
330 |                         historicalFile.statistics.totalLines,
331 |                         historicalFile.statistics.blankLines,
332 |                         historicalFile.commentStats.commentLines});
333 |             }
334 |
335 |             // The last assignment is treated as the current submission.
336 |             const AnalyzedSourceFile &currentFile =
337 |                 analyzedFiles[fileIndices.back()];
338 |
339 |             const IdentifierHistoryResult result =
340 |                 analyzeIdentifierHistory(
341 |                     historicalFiles,
342 |                     currentFile.identifierStats);
343 |
344 |             const CommentHistorySample currentCommentSample{
345 |                 currentFile.statistics.totalLines,
346 |                 currentFile.statistics.blankLines,
347 |                 currentFile.commentStats.commentLines};
348 |
349 |             const CommentHistoryResult commentResult =
350 |                 analyzeCommentHistory(
351 |                     historicalCommentFiles,
352 |                     currentCommentSample);
353 |
354 |             finding.namingEvaluated =
```

The four evaluation/deviation flags

application.cpp: lines 354-363 - Keep not-evaluable separate from evaluated-but-unchanged

```
354 |             finding.namingEvaluated =
355 |                 result.sufficientData && result.stableHistoricalStyle;
356 |             finding.namingDeviation =
357 |                 finding.namingEvaluated && result.reviewRecommended;
358 |             finding.commentingEvaluated =
359 |                 commentResult.sufficientData &&
360 |                 commentResult.stableHistoricalRate;
361 |             finding.commentingDeviation =
362 |                 finding.commentingEvaluated && commentResult.reviewRecommended;
363 |
```

The controller stores a student's history result in StudentStyleFinding. For instance, namingEvaluated is the conjunction of sufficientData and stableHistoricalStyle. namingDeviation adds reviewRecommended to that conjunction. Exactly analogous logic applies to comment history. Print decisions and rate percentages only when the applicable historical evidence is meaningful.

All file pairs and both clone types

Two nested loops traverse i from 0 to fileCount-1 and j from i+1 to fileCount-1. For each pair, findType1BodyMatches and findType2BodyMatches produce separate vectors. Their sizes increment the corresponding aggregate candidate totals. The existing per-type reporters still display detailed evidence during pair iteration. New appendCloneSummaryEntry calls retain a structured copy of each match so the end-of-run report can present it again in one ordered view.

application.cpp: lines 456-489 - Unordered file-pair loops and Type-1 matching

```
456 |         for (std::size_t i = 0; i < analyzedFiles.size(); ++i)
457 |         {
458 |             for (std::size_t j = i + 1; j < analyzedFiles.size(); ++j)
459 |             {
460 |                 const AnalyzedSourceFile &first = analyzedFiles[i];
461 |                 const AnalyzedSourceFile &second = analyzedFiles[j];
462 |
463 |                 const std::string firstRelativePath =
464 |                     first.path.lexically_relative(sourcePath).generic_string();
465 |
466 |                 const std::string secondRelativePath =
467 |                     second.path.lexically_relative(sourcePath).generic_string();
468 |
469 |                 cout << "Comparing files: "
470 |                      << firstRelativePath
471 |                      << " vs "
472 |                      << secondRelativePath << '\n';
473 |
474 |                 std::vector<Type1BodyMatch> matches =
475 |                     findType1BodyMatches(
476 |                         first.tokens,
477 |                         first.functionRegions,
478 |                         second.tokens,
479 |                         second.functionRegions);
480 |
481 |                 aggregate.type1CandidateCount += matches.size();
482 |
483 |                 cout << "Type-1 body-level candidates: "
484 |                      << matches.size() << '\n';
485 |
486 |                 if (!matches.empty())
487 |                 {
488 |                     printType1BodyMatches(
489 |                         cout,
```

application.cpp: lines 512-541 - Type-2-only matching, counting and location retention

```
512 |
513 |                 std::vector<Type2BodyMatch> type2Matches =
514 |                     findType2BodyMatches(
515 |                         first.tokens,
516 |                         first.functionRegions,
517 |                         second.tokens,
518 |                         second.functionRegions);
519 |
520 |                 aggregate.type2OnlyCandidateCount += type2Matches.size();
521 |
522 |                 cout << "Type-2 body-level candidates: "
523 |                      << type2Matches.size() << '\n';
524 |
525 |                 if (!type2Matches.empty())
526 |                 {
527 |                     printType2BodyMatches(
528 |                         cout,
529 |                         firstRelativePath,
530 |                         first.tokens,
531 |                         secondRelativePath,
532 |                         second.tokens,
533 |                         type2Matches);
534 |
535 |                     for (const Type2BodyMatch &match : type2Matches)
536 |                     {
537 |                         if (!appendCloneSummaryEntry(
538 |                                 candidateDetails,
539 |                                 CloneCandidateKind::Type2Only,
540 |                                 firstRelativePath,
541 |                                 first.tokens,
```

Building and printing the final report

application.cpp: lines 568-578 - Pass all previously computed data to the report formatter

```
568 |         // Consolidate already computed evidence for human-readable reporting.
569 |         AnalysisReport report;
570 |         report.studentFolderCount = studentFileIndices.size();
571 |         report.comparedFilePairs = filePairCount;
572 |         report.aggregate = aggregate;
573 |         report.cloneCandidates = candidateDetails;
574 |         report.studentFindings = studentFindings;
575 |
576 |         printCombinedAnalysisReport(cout, report);
577 |
578 |         return 0;
```

The program returns zero after successful directory analysis. In the single-file path (after the directory if-block), it runs the per-file pipeline and returns zero, but does not create student histories or cross-file clone comparisons. An unreadable source file returns 1.

Key architecture explanation
application.cpp is an orchestrator, not a hidden plagiarism classifier. The specialized modules produce stats and candidate records. The final analysis_report reads those records without recomputing them. Explain that division when asked 'which file controls the program?' or 'how did you integrate the features?'

27. Existing modules reused unchanged in this branch

Established module	How Phase 4 depends on it
preprocessor.hpp/.cpp	removeComments(sourceCode) produces the cleaned counterpart required by commenting statistics
tokenizer.hpp/.cpp	tokenize(cleanedCode) yields Token objects with type, lexeme, line and column
code_region.hpp/.cpp	CodeRegion defines beginTokenIndex and exclusive endTokenIndex
function_region.hpp/.cpp	findFunctionBodyRegions(tokens) restricts matching to supported function bodies
type1_clone.hpp/.cpp	findType1BodyMatches returns exact body matches
type2_clone.hpp/.cpp	findType2BodyMatches returns consistent-renaming matches, excluding exact Type-1
type1_report and type2_report	Print the earlier immediate per-pair evidence; new summary is an additional combined view

This is an important implementation-history point: Phase 4 builds on the first three phases; it does not silently claim that Type-1/Type-2 matching was newly invented in this branch. Most feature-branch changes are added data extraction, history and report modules, plus calls from application.cpp.

Full dependency chain in one short example

```
sourceCode = readFile(path);
cleanedCode = removeComments(sourceCode);
commentStats = calculateCommentStyleStatistics(sourceCode, cleanedCode);
tokens = tokenize(cleanedCode);
bodies = findFunctionBodyRegions(tokens);
sourceStats = calculateSourceStatistics(sourceCode, tokens.size(), bodies.size());
nameStats = calculateIdentifierStyleStatistics(tokens);
// Store tokens, bodies, statistics, nameStats, commentStats.
// Later: compare file pairs, build histories and print final report.
```

The snippet above is simplified pseudocode of the actual dependency order; the preceding line-numbered excerpts show the real implementation and exact C++ types.

28. Every new smoke test and exactly what it checks

The feature branch added seven focused C++ smoke-test programs. Each has its own main(), creates controlled inputs, calls one or more production functions, prints a PASS/FAIL label, and returns 0 only if all its assertions pass. They are local regression checks; no automated GitHub CI is claimed here.

Test program	Assertions and protected behavior
identifier_style_smoke	Nine identifier spellings; unique name counting, character total, empty token list, printable labels
identifier_history_smoke	Clear camel-to-snake deviation; stable history; one prior assignment; mixed old styles; too few names; no current dominance
comment_style_smoke	No comments; line/inline; block; multiline block; comment-like text in literal; empty string
comment_history_smoke	Large decrease; stable pattern; one old sample; too few lines; unstable rates; small change; invalid counts; large increase
assignment_order_smoke	Numeric order, valid separators, missing numbers, duplicate numbers, invalid names/suffix/extensions, overflow, empty input
clone_summary_smoke	Type-1/Type-2 entry creation, canonical pair order, invalid boundaries, source-line labels, categories, empty report
analysis_report_smoke	Dataset overview, aggregate statistics, both-side candidate locations, style evidence, human-review union, empty report

The strongest negative tests

- Invalid boundary: CloneSummary rejects CodeRegion{2,4} with a token vector of length 3. It must not dereference tokens[3].
- Assignment overflow: Extremely long assignment numbers must fail validation, not wrap around to a misleading smaller number.
- Mixed histories: Conflicting old conventions or commenting rates must not trigger a misleading deviation judgment.
- Noncode comments: A URL string such as "https://example.com/*text*/" must not be counted as a real comment.
- Empty report: No candidates and no style findings must still produce coherent, non-accusatory output.

PowerShell smoke-test build and run recipes

The commands below illustrate how to compile the focused test translation unit with exactly the implementation files it uses. They are verification commands, not the later documentation-install commands.

```
g++ -std=c++17 -Wall -Wextra -pedantic tests/identifier_style_smoke.cpp
        src/identifier_style.cpp -o identifier_style_smoke.exe
.\identifier_style_smoke.exe
g++ -std=c++17 -Wall -Wextra -pedantic tests/identifier_history_smoke.cpp
        src/identifier_history.cpp -o identifier_history_smoke.exe
.\identifier_history_smoke.exe
```

```
g++ -std=c++17 -Wall -Wextra -pedantic tests/comment_style_smoke.cpp src/comment_style.cpp
        src/preprocessor.cpp -o comment_style_smoke.exe
.\comment_style_smoke.exe
g++ -std=c++17 -Wall -Wextra -pedantic tests/comment_history_smoke.cpp src/comment_history.cpp
        -o comment_history_smoke.exe
.\comment_history_smoke.exe
```

```
g++ -std=c++17 -Wall -Wextra -pedantic tests/assignment_order_smoke.cpp
        src/assignment_order.cpp -o assignment_order_smoke.exe
.\assignment_order_smoke.exe
g++ -std=c++17 -Wall -Wextra -pedantic tests/clone_summary_smoke.cpp src/clone_summary.cpp -o
        clone_summary_smoke.exe
.\clone_summary_smoke.exe
```

```
g++ -std=c++17 -Wall -Wextra -pedantic tests/analysis_report_smoke.cpp src/analysis_report.cpp
        src/clone_summary.cpp -o analysis_report_smoke.exe
.\analysis_report_smoke.exe
$LASTEXITCODE   # check 0 after each run
```

Why tests compile separate executables

A smoke test typically defines its own int main(). The full CloneScope app also defines main() in src/main.cpp. Compiling all tests and the application into a single executable would cause multiple definitions of main. Instead, link each test against only its required implementation modules, then run it independently.

29. The synthetic cohort: every student's expected controls

The 22-student fixture includes shared assignment briefs, differently written implementations, injected clone groups and style controls. The table below reproduces the control categories documented in expected_student_findings.csv and the cohort README. 'Around 30%' is a fixture design label, not the exact rounded rate of every individual source file.

ID	Old naming	Current naming	Name flag	Old rate	New rate	Rate flag	Grp
01	camelCase	snake_case	Yes	~30%	~30%	No	T1
02	snake_case	snake_case	No	~30%	~30%	No	T1

ID	Old naming	Current naming	Name flag	Old rate	New rate	Rate flag	Grp
03	camelCase	camelCase	No	~30%	~30%	No	-
04	camelCase	camelCase	No	~30%	~30%	No	-
05	camelCase	snake_case	Yes	~30%	0%	Yes	T2
06	camelCase	camelCase	No	~30%	~30%	No	-
07	camelCase	camelCase	No	~30%	~30%	No	-
08	camelCase	camelCase	No	~30%	~30%	No	-
09	PascalCase	snake_case	Yes	~30%	0%	Yes	-
10	camelCase	camelCase	No	~30%	~30%	No	-
11	snake_case	snake_case	No	~30%	~30%	No	T2
12	snake_case	snake_case	No	~30%	~30%	No	T2
13	snake_case	snake_case	No	~30%	~30%	No	T2
14	snake_case	snake_case	No	~30%	~30%	No	T2
15	Mixed	snake_case	Not eval.	Mixed	~30%	Not eval.	-
16	camelCase	camelCase	No	0%	0%	No	-
17	snake_case	snake_case	No	~30%	~30%	No	T1
18	snake_case	snake_case	No	~30%	~30%	No	T1
19	snake_case	snake_case	No	~30%	~30%	No	T1
20	camelCase	camelCase	No	0%	~30%	Yes	-
21	camelCase	camelCase	No	~30%	~30%	No	-
22	snake_case	snake_case	No	~30%	~30%	No	-

In the last column, T1 means an injected Type-1 prime-checking relationship, T2 means an injected Type-2-only Fibonacci relationship, and - means no intentionally injected clone. This table is the authored fixture control map, not a real-world misconduct register.

Who is actually injected into the clone groups?

Group	Exact pairs (10 per group)
Type-1: prime checking	01-02, 01-17, 01-18, 01-19, 02-17, 02-18, 02-19, 17-18, 17-19, 18-19
Type-2: Fibonacci	05-11, 05-12, 05-13, 05-14, 11-12, 11-13, 11-14, 12-13, 12-14, 13-14

For the prime group, all matched body locations printed at assignment3.cpp lines 4-29. For the renamed Fibonacci group, source-side line ranges are 34-64 for students 05, 11 and 13, and 32-62 for students 12 and 14. These are the observed line ranges, not inferred from group membership.

Four CSV controls and what each contains

Filename	Purpose
manifest.csv	66 rows mapping student ID, assignment number/path, brief, algorithms, naming and commenting design, injected clone-group label
expected_clone_pairs.csv	20 intentionally injected expected pair relationships with clone category and shared algorithm
expected_student_findings.csv	22 rows listing expected history/current naming/commenting controls and deviations
sample_inputs.csv	One standard-input string per assignment with expected categories of output

The standard examples are: assignment1 input 5 42 30 1221; assignment2 input 9 2 7 1 5 3 7; assignment3 input 29 12 3 4 5 -2. Each independent program can be compiled and executed with its own main(). The fixture was constructed to permit deterministic expected behavior while resembling introductory C++ coursework.

30. Working through the observed console report

File-pair arithmetic

The directory controller uses i<j. With 66 analyzed files, there are 66 choices for i but only 65 distinct partners for each file on average before dividing double counts: 66×65/2 = 2,145 unordered pairs. This includes same-student comparisons, cross-assignment comparisons and across-student comparisons. The program does not restrict comparisons to matching assignment numbers.

Recognized bodies versus candidate pairs

The controlled run reports 242 supported function bodies across all 66 files. For one file pair with r bodies in the first and s in the second, a matcher may examine up to r×s region combinations. That is a different counting unit from the 2,145 source-file pairs and the 20 matching body-region pairs.

Expected report totals

Part of console report	Observed summary
Overview	22 student folders; 66 files; 2,145 file pairs
Source code	6,973 lines; 242 blank lines; 30,995 tokens; 242 supported bodies
Clone groups	10 Type-1 prime pairs; 10 Type-2-only Fibonacci pairs
Naming history	3 deviations; 21/22 evaluated; student15 not evaluable
Commenting history	3 deviations; 21/22 evaluated; student15 not evaluable
Human review	12 unique students with at least one indicator

A combined signal example

student05 has a Fibonacci Type-2-only candidate against students 11-14; its historical camelCase convention shifts to snake_case; and its commenting rate drops from a stable old pattern to zero. The final report lists all three reasons under one student ID. Conversely, student09 has naming and commenting changes but no injected clone; the report must preserve those independent signals without falsely inventing a clone.

A deliberately unevaluable student

student15's prior naming/commenting history is mixed. There is no consistent baseline for a strict change judgment. Thus the report shows student15 among 'Naming histories not evaluable' and 'Commenting histories not evaluable', rather than including student15 in a list of cleared or suspicious students.

31. Limitations, possible errors and design trade-offs

Boundary	Concrete effect and honest explanation
Restricted parser	Function discovery uses tokens and brace heuristics, so advanced C++ constructs may not produce reliable body regions
Only Type-1/2 now	Reordered statements, inserted/deleted logic or semantically equivalent algorithms are not covered by the implemented comparators
Restricted Type-2	Only identifier lexemes may change consistently; changed literals, operators or token structure reject a match
Scope-unaware matching	Identifiers are compared lexically, not through AST symbol binding or variable-shadowing resolution
Comment scanner limits	Basic strings/chars are recognized, but raw strings, complex preprocessing and malformed source may be outside supported behavior
Stylistic heuristics	Naming and comment thresholds were selected for an educational prototype, not learned or statistically calibrated against authors
Chronology from filename	A valid numbered assignment name is required; the system does not inspect submission timestamps or learning-management metadata
All-pairs workload	With n files and multiple functions per file, runtime can grow substantially; the 66-file run is not an asymptotic performance benchmark
No automated verdict	Standard algorithms, common teaching examples and templates can create genuine token-level matches without misconduct
No structured export yet	CSV/JSON serialization is the next proposed branch, and is not a current implementation claim

Suggested improvements, not claims of work already completed

- Add CSV/JSON serialization of the already constructed AnalysisReport data, preserving both-side source lines and evaluation states.
- Create end-to-end automated tests comparing machine-readable exported results with the existing control CSVs.
- Consider limiting expensive comparisons to matching assignments in a future optional mode, while documenting that the current tool still compares all pairs.
- Evaluate larger, genuinely permissioned and anonymized student samples before making external accuracy claims.
- Extend function parsing and add Type-3/Type-4 algorithms only after the current prototype is stable and the midterm review is complete.

32. Extended viva preparation: defend every design choice

Advanced Q01: Why was the source-statistics function separated from the application controller?

It has one responsibility, can be tested independently, and receives explicit token/region counts rather than having to reproduce tokenization.

Advanced Q02: Why preserve original and cleaned source simultaneously?

The cleaned string is useful for token matching, while the original contains comment information and physical lines that must not be lost.

Advanced Q03: What does an unordered_set buy us in identifier statistics?

Average fast insert/lookup and a boolean inserted result, letting one identifier spelling contribute once regardless of how many times it is reused.

Advanced Q04: Why exclude single-letter and all-lowercase names from historical conventions?

They provide weak evidence for a multiword naming convention; the history module intentionally evaluates only three stronger style categories.

Advanced Q05: Why require a style to account for 80% of names?

It avoids treating a narrow plurality as a reliable dominant convention; this is a heuristic design threshold, not a probabilistic confidence score.

Advanced Q06: Is a naming style change enough to conclude a clone?

No. The naming-history module does not compare files; it only recommends manual inspection when a stable old convention clearly changes.

Advanced Q07: How does comment history avoid small sample fluctuations?

It requires at least 20 nonblank lines per sample, two historical files and sufficiently consistent historical rates.

Advanced Q08: Why pool old comment rates by line counts?

Summed comments divided by summed nonblank lines gives each old source line one weight instead of treating short and long assignments identically.

Advanced Q09: Why does the code check blankLines <= totalLines?

Otherwise the denominator computation on an unsigned size_t could underflow and produce an invalid rate.

Advanced Q10: Why does assignment_order use std::map rather than a vector sort?

It simultaneously enforces unique numeric suffix keys and produces ascending numerical order without a separate comparator.

Advanced Q11: Why reject two different filenames with the same assignment number?

The program cannot know which of those is the intended current submission; a silent selection would corrupt the student's historical baseline.

Advanced Q12: Why use the exclusive end token index?

It makes token intervals compatible with standard half-open C++ ranges; the final included token is endTokenIndex-1.

Advanced Q13: Why does clone_summary swap line ranges when swapping paths?

A source line range belongs to its specific file; swapping only the filenames would attach incorrect evidence to the wrong student.

Advanced Q14: What does deterministic candidate presentation mean?

Sorting candidate kind, relative paths and positions ensures the same report order even if source discovery order changes.

Advanced Q15: Why does analysis_report use map<string,set<string>>?

It guarantees one student row with unique reasons regardless of how many matching pairs involve that student.

Advanced Q16: Does the final report calculate new clone results?

No. It consumes already computed AggregateStatistics, CloneSummaryEntry and StudentStyleFinding records, preventing duplicate detection logic.

Advanced Q17: Why are 20 candidate pairs not 20 students?

Five-person groups each yield ten unordered pairwise combinations; the same individuals recur in multiple pairs. Review students are a separate set union.

Advanced Q18: What proves the full application integration works on the fixture?

The observed 22/66 overview, 2,145 pair count, 10+10 candidates, three naming and three commenting deviations, and detailed line locations match documented synthetic controls.

Advanced Q19: Can you quote precision/recall or call the dataset real?

No. This controlled synthetic fixture supplies regression ground truth only. It is not representative of actual submissions and does not support population-level accuracy claims.

Advanced Q20: Which module should a CSV/JSON exporter depend on?

The already assembled AnalysisReport record is the clean boundary; serialize data and do not repeat tokenization, clone matching or history analysis.

33. Study checklist and milestone sign-off

Checkpoint	What you should be able to demonstrate or explain
Architecture	Draw main -> runCloneScope -> per-file stages -> historical analysis and clone pair loops -> AnalysisReport
Module contracts	Name each added .hpp public type/function and identify the .cpp implementation
Threshold math	Compute a dominant naming fraction, pooled comment rate and deviation threshold on paper
File order	Explain why assignment10 is current after assignment1 and assignment2, even if discovered out of order
Candidate evidence	Trace a pair of source-line ranges back to tokenizer token.line and CodeRegion bounds
Stable report	Explain why students are deduplicated and why one style-only student can be flagged
Testing	Run the seven new smoke suites, then the 22-student integrated fixture; state what each validates
Honest limits	State that Type-3/4 and CSV/JSON are not yet implemented, and that a match is not a misconduct verdict
Documentation	Review this draft, then update README.md, add the approved PDF, and commit it on the current feature branch

You are ready when...
You can independently explain the order of every module call, each struct's purpose, all three history states, why there are 2,145 file pairs and 20 body-level candidate pairs, and why student09 has style findings without a clone candidate. Then we polish the README, finalize this PDF, and prepare the PR review.